SC-FF Final Project - Group 3

J. Elefante, A. Lo Bello, G. Sala

December 2025

Introduction

Midori64 [1] is a lightweight block cipher, optimized with respect to energy consumption in the circuit needed for implementation. To achieve this certain design choices were made, such as the use of an almost MDS binary matrix (over a MDS one) or an optimal cell-permutation layer to find a concrete balance between security and energy usage.
The cipher uses an SPN architecture, has a 64 block size, accepts 128-bit keys and runs for 16 rounds. It was later discovered [2] that this cipher admits a class of weak keys that can be identified with a single query and for which an invariant subspace attack is feasible.
Our assignment was to implement in MAGMA both an encryption and decryption function for such cipher as well as the invariant attack presented in the cited papers, while also documenting our progress in a lab report.
We discussed together as a group all the theory behind the cipher and the possible implementations, to then divide the job of actually writing the parts as:
- J. Elefante: Encryption and Decryption function.
- A. Lo Bello: Invariant Subspace Attack.
- G. Sala: Lab Report.

Encryption-Decryption

A brief description of the cipher is given in this section.

Encryption

As stated earlier, Midori64 was designed following a Substitution Permutation Network, data is stored in a 4x4 array S where each entry is a cell of 4 bits (nibble) and is therefore represented by a hexadecimal number. This representation will be used during the whole report, and when met with a sum between hexadecimal numbers one should think of a bitwise XOR between nibbles.

$$S := \begin{bmatrix} s_0 & s_4 & s_8 & s_{12} \\ s_1 & s_5 & s_9 & s_{13} \\ s_2 & s_6 & s_{10} & s_{14} \\ s_3 & s_7 & s_{11} & s_{15} \end{bmatrix}$$

The base elements of the cipher are:

- **Cell Substitution** (Non-linear involution)
  The following substitution is applied to each cell s_i.

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
C	A	D	3	E	B	F	7	8	9	1	5	0	2	4	6

  This is the only non-linear component of the cipher.

- **Cell Shuffle** (Linear non-involutory bijection)
  The following permutation is applied to each cell s_i.

  $(s_0, s_1, ..., s_{15}) \leftarrow (s_0, s_{10}, s_5, s_{15}, s_{14}, s_4, s_{11}, s_1, s_9, s_3, s_{12}, s_6, s_7, s_{13}, s_2, s_8)$.

- **Column Mix** (Linear involution)
  The following almost maximum distance separable matrix is applied to each column of the state array S.

$$M := \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

That is:
$(s_i, s_{i+1}, s_{i+2}, s_{i+3})^T \leftarrow M(s_i, s_{i+1}, s_{i+2}, s_{i+3})^T, \quad i = 0, 4, 8, 12,$
where, for every hexadecimal number a, the only multiplications are $0 \cdot a = 0$ and $1 \cdot a = a$.

- **Key Addition** (Linear involution)
  A round key is XORed to the state S.
  The round key generation takes as input the 128-bit master key K, splits it into two 64-bit keys as $K = K_0 || K_1$. A whitening key, used before the rounds, is generated as $WK = K_0 \oplus K_1$. The rounds key are instead $RK_i = K_{(i \mod 2)} \oplus \alpha_i$, with α_i being a seemingly random round constant, expressed below as matrices for simplicity.

i	α_i	i	α_i	i	α_i	i	α_i
0	0 0 1 0 0 1 0 0 0 0 1 1 1 1 1 1	1	0 1 1 0 1 0 1 0 1 0 0 0 1 0 0 0	2	1 0 0 0 0 1 0 1 1 0 1 0 0 0 1 1	3	0 0 0 0 1 0 0 0 1 1 0 1 0 0 1 1
4	0 0 0 1 0 0 1 1 0 0 0 1 1 0 0 1	5	1 0 0 0 1 0 1 0 0 0 1 0 1 1 1 0	6	0 0 0 0 0 0 1 1 0 1 1 1 0 0 0 0	7	0 1 1 1 0 0 1 1 0 1 0 0 0 1 0 0
8	1 0 1 0 0 1 0 0 0 0 0 0 1 0 0 1	9	0 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0	10	0 0 1 0 1 0 0 1 1 0 0 1 1 1 1 1	11	0 0 1 1 0 0 0 1 1 1 0 1 0 0 0 0
12	0 0 0 0 1 0 0 0 0 0 1 0 1 1 1 0	13	1 1 1 1 1 0 1 0 1 0 0 1 1 0 0 0	14	1 1 1 0 1 1 0 0 0 1 0 0 1 1 1 0	15	0 1 1 0 1 1 0 0 1 0 0 0 1 0 0 1

The pseudo-MAGMA-code to describe the implementation is:

```
MIDORI64_enc()
KeyAddition();
for i in [0..14] do
   CellSubstitution();
   CellShuffle();
   ColumnMix();
   KeyAddition();
end for;
CellSubstitution();
KeyAddition();
```

Decryption

Given the nature of block ciphers designed with the SPN, once the encryption function is implemented, it's just a matter of reversing the functions and order of operations, therefore we report here only the pseudocode needed:

```
MIDORI64_dec()
KeyAddition();
for i in [14..0] do
   CellSubstitution();
   ColumnMix();
   CellInverseShuffle();
   KeyAddition*();
end for;
CellSubstitution();
KeyAddition();
```

Remark: The decryption is not the perfect inversion (in terms of implementation) of the encryption. In fact, we use *KeyAddition*()* which is not a XOR with the *ith*-round key, but rather with its image under CellInverseShuffle() $\circ$ ColumnMix(). This is done to make the hardware implementation essentially the same for both encryption and decryption and of course leads to the same result, thanks to the linearity of CellInverseShuffle() and ColumnMix().

Implementation

The implementation for this cipher was rather simple, as one would expect from an SPN block cipher. We started by defining some helper functions to aid in the conversion of the data used in the cipher, to then easily define those in charge of applying the S-boxes, P-boxes, Column Mix and retrieving the *ith*-round key. MIDORI64_enc(P, K) is just the above functions called as in the already seen pseudocode. With a similar reasoning MIDORI64_dec(C, K) was also implemented.

Invariant Subspace Attack

Mathematical Analysis

An invariant subspace attack is feasible when a given cipher maps a proper subspace $\mathbb{S}$ of the plaintext space $\mathcal{P}$ into another proper subspace of the ciphertext space $\mathcal{C}$ for a certain set $\mathbf{K}$ in the key space $\mathcal{K}$, leading to a simplified version of the cipher. Although not common, when this type of attack is implementable, a class of weak plaintexts and/or keys is found, highlighting possible flaws for the real world application of the cipher.
In our case, [2] found that Midori64 has a class of 2^{32} weak keys for which an almost immediate key-recovery attack is implementable with just two suitable pairs of plaintext-ciphertext. In our case, the weak set of keys is $\mathbf{K} = \{0,1\}^{32} = \mathbb{K}^2$, where $\mathbb{K} = \{0,1\}^{16}$, and the affine subspace is $\mathbb{S} = \{8,9\}^{16}$. Formally:

Theorem. *[2]*
If the 128-bit master key $K_0||K_1$ satisfies $K_0, K_1 \in \mathbb{K}$, then any plaintext $P \in \mathbb{S}$ is mapped by Midori64 to a ciphertext $C \in \mathbb{S}$ with probability one.

With the above, one would be able to recognize with a non-negligible probability (in an attack-game scenario) when a weak key is being used with a single query and then look for a key-recovery attack in the newly smaller cipher.
The theorem follows from the fact that $\mathbb{S}$ is mapped into itself in *every* operation. In fact:

- $\mathbb{S} \leftarrow \text{CellSubstitution}(\mathbb{S})$, since 8 and 9 are fixed points of the S-box;
- $\mathbb{S} \leftarrow \text{CellShuffle}(\mathbb{S})$, since this is just a permutation;
- $\mathbb{S} \leftarrow \text{ColumnMix}(\mathbb{S})$, since $a, b, c \in \{8,9\} \implies (a+b+c) \in \{8,9\}$;
- $\mathbb{S} \leftarrow \text{KeyAddition}(\mathbb{S})$, because $K_0, K_1 \in \mathbb{K} \implies WK,\ RK_i \in \mathbb{K}$, and then because $8+1 = 9+0 = 9$ and $8+0 = 9+1 = 8$.

Luckily (or unluckily, depending on who you're talking with), the S-box used in Midori64 (which is the only non-linear function of the cipher), has both 8 and 9 as fixed points of the function, so the cipher over $\mathbb{S}$ becomes fully linear, with all the intermediate states in $\mathbb{S}$. Given then one pair (P, C) of plaintext-ciphertext in $\mathbb{S}$, it is possible to follow the whole path of P through the now linear cipher and write the following linear system:

$$
\begin{aligned}
k_0 \oplus k_{11} \oplus k_{14} \oplus k_{15} \oplus k_{21} \oplus k_{22} \oplus k_{23} \oplus k_{26} \oplus k_{28} \oplus k_{29} \oplus k_{30} \oplus k_{31} &= p_0 \oplus p_5 \oplus p_6 \oplus p_7 \oplus p_{10} \oplus p_{11} \oplus p_{12} \oplus p_{13} \\
&\oplus c_5 \oplus c_6 \oplus c_7 \oplus c_{10} \oplus c_{12} \oplus c_{13} \oplus c_{14} \oplus c_{15} \\
k_1 \oplus k_{11} \oplus k_{19} \oplus k_{24} \oplus k_{26} \oplus k_{29} \oplus k_{31} &= p_1 \oplus p_3 \oplus p_8 \oplus p_{10} \oplus p_{11} \oplus p_{13} \oplus p_{15} \\
&\oplus c_3 \oplus c_8 \oplus c_{10} \oplus c_{13} \oplus c_{15} \oplus 1 \\
k_2 \oplus k_{14} \oplus k_{19} \oplus k_{21} \oplus k_{22} \oplus k_{23} \oplus k_{24} \oplus k_{28} \oplus k_{30} \oplus k_{31} &= p_2 \oplus p_3 \oplus p_5 \oplus p_6 \oplus p_7 \oplus p_8 \oplus p_{12} \oplus p_{15} \\
&\oplus c_3 \oplus c_5 \oplus c_6 \oplus c_7 \oplus c_8 \oplus c_{12} \oplus c_{14} \oplus c_{15} \\
k_3 \oplus k_{15} \oplus k_{19} \oplus k_{24} \oplus k_{25} \oplus k_{29} &= p_8 \oplus p_9 \oplus p_{13} \oplus p_{15} \oplus c_3 \oplus c_8 \oplus c_9 \oplus c_{13} \oplus 1 \\
k_4 \oplus k_{11} \oplus k_{13} \oplus k_{15} \oplus k_{22} \oplus k_{25} \oplus k_{27} \oplus k_{28} \oplus k_{29} \oplus k_{30} &= p_4 \oplus p_6 \oplus p_9 \oplus p_{12} \oplus p_{14} \oplus p_{15} \\
&\oplus c_6 \oplus c_9 \oplus c_{11} \oplus c_{12} \oplus c_{13} \oplus c_{14} \oplus 1 \\
k_5 \oplus k_{14} \oplus k_{22} \oplus k_{23} \oplus k_{25} \oplus k_{28} \oplus k_{29} \oplus k_{30} &= p_5 \oplus p_6 \oplus p_7 \oplus p_9 \oplus p_{12} \oplus p_{13} \\
&\oplus c_6 \oplus c_7 \oplus c_9 \oplus c_{12} \oplus c_{13} \oplus c_{14} \oplus 1 \\
k_6 \oplus k_{13} \oplus k_{14} \oplus k_{15} \oplus k_{22} \oplus k_{25} \oplus k_{28} \oplus k_{29} &= p_9 \oplus p_{12} \oplus p_{14} \oplus p_{15} \oplus c_6 \oplus c_9 \oplus c_{12} \oplus c_{13} \\
k_7 \oplus k_{13} \oplus k_{14} \oplus k_{15} \oplus k_{23} &= p_{13} \oplus p_{14} \oplus p_{15} \oplus c_7 \\
k_8 \oplus k_{15} \oplus k_{24} \oplus k_{29} &= p_{13} \oplus p_{15} \oplus c_8 \oplus c_{13} \\
k_9 \oplus k_{11} \oplus k_{13} \oplus k_{14} \oplus k_{24} \oplus k_{28} &= p_8 \oplus p_9 \oplus p_{11} \oplus p_{12} \oplus p_{13} \oplus p_{14} \oplus c_8 \oplus c_{12} \\
k_{10} \oplus k_{11} \oplus k_{25} &= p_9 \oplus p_{10} \oplus p_{11} \oplus c_9 \oplus 1 \\
k_{12} \oplus k_{13} \oplus k_{14} \oplus k_{15} \oplus k_{29} &= p_{12} \oplus p_{14} \oplus p_{15} \oplus c_{13} \\
k_{16} \oplus k_{19} \oplus k_{24} \oplus k_{25} \oplus k_{29} \oplus k_{31} &= p_0 \oplus p_3 \oplus p_8 \oplus p_9 \oplus p_{13} \oplus p_{15} \\
&\oplus c_0 \oplus c_3 \oplus c_8 \oplus c_9 \oplus c_{13} \oplus c_{15} \oplus 1 \\
k_{17} \oplus k_{19} \oplus k_{22} \oplus k_{23} \oplus k_{24} \oplus k_{25} \oplus k_{26} \oplus k_{27} \oplus k_{28} \oplus k_{31} &= p_1 \oplus p_3 \oplus p_6 \oplus p_7 \oplus p_8 \oplus p_9 \oplus p_{10} \oplus p_{11} \\
&\oplus p_{12} \oplus p_{15} \oplus c_1 \oplus c_3 \oplus c_6 \oplus c_7 \oplus c_8 \oplus c_9 \\
&\oplus c_{10} \oplus c_{11} \oplus c_{12} \oplus c_{15} \\
k_{18} \oplus k_{19} \oplus k_{21} \oplus k_{22} \oplus k_{23} \oplus k_{24} \oplus k_{28} \oplus k_{29} \oplus k_{30} \oplus k_{31} &= p_2 \oplus p_3 \oplus p_5 \oplus p_6 \oplus p_7 \oplus p_8 \oplus p_{12} \oplus p_{13} \\
&\oplus p_{14} \oplus p_{15} \oplus c_2 \oplus c_3 \oplus c_5 \oplus c_6 \oplus c_7 \oplus c_8 \\
&\oplus c_{12} \oplus c_{13} \oplus c_{14} \oplus c_{15} \oplus 1 \\
k_{20} \oplus k_{22} \oplus k_{23} \oplus k_{25} \oplus k_{28} \oplus k_{29} &= p_4 \oplus p_6 \oplus p_7 \oplus p_9 \oplus p_{12} \oplus p_{13} \\
&\oplus c_4 \oplus c_6 \oplus c_7 \oplus c_9 \oplus c_{12} \oplus c_{13},
\end{aligned}
$$

Figure 1: Linear System of $\mathbb{S}$

Where $K_0 = k_0, ..., k_{15}$; $K_1 = k_{16}, ..., k_{31}$; $P = p_0, ..., p_{15}$; $C = c_0, ..., c_{15}$; all nibbles.
This system has 16 equations in 32 (k_i) unknowns and is therefore undetermined. Given the shape of the system, we are able to simply enunciate the values of the same 16 nibbles ($k_{11}, k_{13}, k_{14}, k_{15}, k_{19}, k_{21}, k_{22}, k_{23}, k_{24}, k_{25}, k_{26}, k_{27}, k_{28}, k_{29}, k_{30}, k_{31}$) and obtain the rest with simple operations without explicitly calling a function. For the actual key-recovery attack one would need a second pair (P_1, C_1), guess 16 nibbles of the weak key, solve the system to retrieve the other half, build the guessed key GK and then check if MIDORI64_enc$(P_1, GK) = C_1$. This results in a brute force attack on a space with size 2^{16}, way smaller than 16^{32}, of the key space.
Remark: The attack is possible because of the unfortunate choices made during the design; as also stated in [2], the easiest way to fix this problem would be by changing the round constants into something with some nibbles different from $0, 1$.

Implementation

In the first part of MIDORI64_KeyRecovery(P, C, P_1, C_1), we solve the linear system 1 simply by defining the two matrices A,B s.t $A \cdot k^T = B$, where $A \in Mat_{16,32}(\{0,1\})$, $B \in Mat_{16,1}(\{8,9\})$ whose entries are nibbles, and k being the vector of unknowns nibbles of the key. Once the linear system is solved, the brute force is simply a for cycle that returns the guessed key GK as soon as we get the match MIDORI64_enc$(P_1, GK) = C_1$.
MIDORI64_SubspaceAttack(K) just defines the two plaintexts (and therefore the two ciphertexts) needed for the previous function, and returns the result of

our attack. We decided to solve the system this way, instead of the "substitution" discussed above, because in MAGMA the function *Solution* can directly return the kernel of such system, which means that the brute force section of the code looks more clear and compact. Since most of the time is spent by repeatedly calling the encryption function, our choice does not provoke a noticeable increase in running time. If storage were a problem, the other way would have to be implemented.

Remark: In MIDORI64_SubspaceAttack, P (and consequently C) *must* belong to $\mathbb{S}$ while P_1 (and consequently C_1) *must not* belong to $\mathbb{S}$.
This is because P is used for building the linear system, which requires the plaintext to be in the correct subspace. P_1, which is used for the brute-force attack, must be taken outside of $\mathbb{S}$. Every cipher can be described as a system of equations, and when a brute-force attack is initiated we are basically trying out all of the possible values of the unknowns (key bits) and see which of them are a solution for such system. In our case the attack is carried out on *all* the solutions of the (smaller) linear system, which are satisfied by *all* the pairs $(P', C') \in \mathbb{S} \times \mathbb{S}$, so, by taking $P_1 \in \mathbb{S}$, we are effectively checking if the solutions of the system from (P, C) are also solution from (P', C'), but both being 1 means this is trivially true. To see this, one can also slightly modify our code so that MIDORI64_KeyRecovery*(P, C, P_1, C_1) returns the number of times that a match is found in the brute-force section and verify that indeed, $\forall P, P', C = Enc_K(P), C' = Enc_K(P')$ one has MIDORI64_KeyRecovery*$(P, C, P_1, C_1) = 2^{16}$.

Remark: $|\mathcal{K}| = 16^{32} > 16^{16} = |\mathcal{P}| = |\mathcal{C}|$, so there are different keys that send the same plaintext P into the same ciphertext C, meaning that a priori our brute-force attack might not return the correct key. However, we are only checking 2^{16} keys, way less than the entire set, and if we treat the cipher as a random function, the probability that at least one different key from the correct one maps P_1 into C_1 is $1 - P_{bin}(0; 2^{16} - 1, 16^{-16}) = 3.553 \cdot 10^{-15} << 1$. Being this low, our attack will have success most of the times. One can also slightly modify our code to let the KeyRecovery attack store all the guessed keys for which the match happens, and throw an error if more than one is stored, but this would lead to an increase in running time as every key has to be checked once, which we decided to not implement.

Bibliography

[1] S. Banik et al. "Midori: A Block Cipher for Low Energy". In: *Advances in Cryptology – ASIACRYPT 2015*. Ed. by T. Iwata and J. H. Cheon. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 411–436.

[2] J. Guo et al. *Invariant Subspace Attack Against Midori64 and The Resistance Criteria for S-box Designs*. Cryptology ePrint Archive, Paper 2016/973. 2016.